AI TEAM SELECTOR

Five AI Algorithms Implemented From Scratch in Pure Python

Emad Yar Khan 31651

Fahad Azfar 31659

Fahad Faisal 31583

Muhammad Saad Godil 31616

PROJECT LINK

GitHub Repository: *https://github.com/Fahad2120/EFSF-AI-PROJECT*

ABSTRACT

This report presents the design, implementation, and evaluation of an AI Team Selector — a full-stack cricket intelligence system built exclusively on Pakistan Super League (PSL) match data, specifically featuring only Pakistani native cricketers. The system recommends an optimal 11-player squad for a specified PSL venue by passing player statistics through a sequential pipeline of five machine learning algorithms, all implemented from scratch in pure Python without any external ML libraries such as scikit-learn or TensorFlow.

The pipeline begins with Linear Regression using gradient descent to compute impact scores, followed by K-Means Clustering with k-means++ initialisation to group players into tactical archetypes. A Naive Bayes classifier with Laplace smoothing then estimates venue-specific performance probabilities, while a CART Decision Tree assigns interpretable performance tiers. Finally, a Genetic Algorithm with elitism and tournament selection searches the combinatorial space of team compositions to converge on the highest-scoring, role-balanced eleven.

The updated system features a fully expanded frontend with five distinct pages: a Team Selector, a Player Explorer with AI Form Predictions, a Leaderboard of Top Performers segmented by role, a Statistics dashboard with interactive charts, and a Player Detail view. The backend API has been extended with year-range filtering, full-stats endpoints, and alternative player suggestions. All player data remains restricted to Pakistani cricketers with PSL appearances.

Keywords: *genetic algorithm, linear regression, K-Means, Naive Bayes, CART decision tree, player analytics, AI predictions*

TABLE OF CONTENTS

1. INTRODUCTION

Cricketteam selection isone of the most consequential decisions in professional cricket. At the T20 level — and specifically within the Pakistan Super League (PSL) — the pressure is amplified by fast-paced match conditions, venue-specific pitch behaviour, and a shallow margin for error across just 20 overs. Historically, team selection has been a process governed by intuition, scouting reports, and selector experience. While these human elements remain valuable, they are increasingly being complemented, and in some cases challenged, by data-driven approaches.

The AI Team Selector is a software system purpose-built to support PSL team selection decisions using statistical machine learning. The system operates on a dataset drawn exclusively from PSL match records, limiting its scope to Pakistani cricketers. This restriction is deliberate: it ensures that all recommendations are grounded in domestic T20 performance rather than conflating international or foreign player statistics that would distort the analytical model for a PSL context. Additionally, examining domestic statistics allows the system to surface future and young upcoming prospects in the league.

What distinguishes this project from similar analytics tools is the decision to implement every machine learning algorithm entirely from scratch in pure Python. No external ML frameworks such as scikit-learn, PyTorch, XG Boost and etc, are used in the model pipeline.

1.1Objectives

The project was designed around four core objectives:

- Implement five distinct AI algorithms from scratch:
  1. Linear Regression
  2. K-Means Clustering
  3. Naive Bayes
  4. CART Decision Tree
  5. Genetic Algorithm

Each serving a specific role in the player evaluation and team selection pipeline.

- Compare different data: compare different sources of data to show which one is better. This was done by integrating venue intelligence into the scoring model, so that the system's recommendations are sensitive to the different pitch and ground conditions at different PSL venues (across Pakistan only), Karachi, Lahore, Multan, and Rawalpindi. Alongside data fields associated with each player, for example, strike rate or wickets taken.

- Explore corner cases: Study extreme cases which are commonly overlooked. These can be cases for which people usually assume there will not occur. But they can occur occasionally and can have adverse effects. In other words, you may want to study a method by removing a common assumption. Players who have inconsistent data were blacklisted. For example, Amad Butt, Nahid Rana and etc, were removed in order to get better results.
- Identify root cause of an effect: you may want to study why a particular result is observed in an application. If an algorithm performs poorly or well in some scenarios, then identify the factor which influences its performance. To ensure proper and reliable results, we added a parameter that prevents players who have played less than 5 matches in the PSL from being added in the team.
- Produce a system that is not only accurate but interpretable — every selected player is accompanied by a human-readable justification derived from their statistics, rather than an opaque model score.

1.2 ScopeandConstraints

The system is intentionally scoped to the PSL and to Pakistani cricketers. Specifically:

- All player statistics come from PSL match records loaded from Excel files using Pandas and OpenPyXL.
- Players with fewer than five PSL matches are excluded from selection consideration to ensure statistical significance.
- A defined blacklist removes players whose data quality is insufficient *(e.g., Bilawal Bhatti, Nahid Rana, Amad Butt, Mohammad Imran (1))*.
- Venue compositions are hardcoded based on domain expertise: Rawalpindi, for instance, favours four fast bowlers due to its pace-friendly surface, while Karachi allocates slots for two spin bowlers.

1.3 SystemOverview

At a high level, the system takes a venue name as input and returns a recommended 11-player squad with role assignments and selection reasons. Internally, this involves seven sequential stages:

- Data loading and enrichment
- Feature extraction and normalisation
- Impact scoring via Linear Regression
- Player clustering via K-Means
- Probabilistic venue scoring via Naive Bayes
- Tier classification via CART Decision Tree
- Final team assembly via Genetic Algorithm

The system is deployed asaReact/TypeScript single-page application with five pages — Team Selector, Player Explorer, Leaderboard,Statistics, and Player Detail — all served by a FastAPI Python backend. Each pipeline stage is explainedindetail in Section 3. Frontend features added in the current version are documented in Section 4.

2. RELATED WORK

Theintersectionofmachine learning and cricket analytics has grown substantially over the past decade, driven by the availability of structured ball-by-ball and innings-level datasets from T20 leagues worldwide.

2.1Player Performance Modelling

Early quantitative approaches to cricket performance focused on aggregating career statistics into composite indices. Lemmer (2004) proposed batting performance indices that weight averages against strike rate, establishing a precedent for numeric scoring of batters that remains relevant to modern approaches. The AI Team Selector's linear regression component directly extends this concept by learning optimal weights across six features using gradient descent, rather than assigning them by hand.

More recent work by Saikia, Bhattacharjee, and Lemmer (2016) applied machine learning to T20 bowler performance prediction using classification models trained on career and recent-form features. Their finding that recent form over the last five matches is a stronger predictor of immediate performance than career averages directly influenced the *_recent_form()* function in this system, which restricts its calculation to the five most recent match records per player.

2.2TeamSelectionas CombinatorialOptimisation

The problem of selecting an optimal 11-player cricket squad from a pool of candidates is a constrained combinatorial optimisation problem. Ahmed, Deb, and Jindal (2013) demonstrated that multi-objective genetic algorithms significantly outperform greedy heuristics for ODI cricket squad selection when role-balance constraints are imposed. Their work provided the theoretical justification for the Genetic Algorithm approach used in this project, particularly the use of role-partitioned chromosome pools and repair functions to maintain feasibility across crossover operations.

2.3Venue and Condition Modelling

The influence of playing surface on match outcomes has been documented extensively. Manage and Scariano (2013) used principal component analysis on cricket ground data to show that venue characteristics explain a statistically significant portion of batting and bowling performance variance. This insight underpins the venue-weighting mechanism in this system, where ground-specific batting averages, strike rates, bowling economies, and wicket rates are incorporated into the final scoring formula with a 30% weight allocation.

2.4Probabilistic Classification inSports

Naive Bayes classifiers have been applied to sportsoutcome prediction in several domains due to their computational efficiency and strong performance under feature independence assumptions. Kampakis and Thomas (2015) demonstrated competitive accuracyfor cricket match outcome prediction using a Naive

Bayes baseline. In the present system, Naive Bayes is applied not to match outcomes but to player-level venue performance classification, estimating the probability that a player performs well, average, or poorly at a given venue given their recent form metric.

2.5Positioningofthis Work

What distinguishes the AI TeamSelector from prior work is the integration of all five algorithm types — regression, clustering, probabilistic classification, decision trees, and evolutionary optimisation — into a single, coherent, venue-awarepipeline applied exclusively to PSL data. Previous systems typically apply one or two techniques in isolation.The from-scratch implementation also ensures full transparency of the mathematical operations, withnoblack-box library dependencies.

3. METHODOLOGY

Thissection details each component of the AI pipeline. The pipeline consists of seven sequential stages that transform raw PSL match records into a final recommended 11-player squad. Each stage is described with reference to its implementation in ai_engine.py.

3.1Data LoadingandPlayerEnrichment

The system loads five datasets from PSL Excel files: masterRecords (innings-level match records), playerRoles (career aggregates per player), groundBatting and groundBowling (venue-specific aggregates), allRounders, wicketkeepers, and bowlers. Each player record is initialised from playerRoles and enriched progressively. The bowlers dataset adds bowling type and updates economy and wickets-per-match. Wicketkeeper and all-rounder datasets override role classifications. Recent form is computed from the five most recent masterRecords entries per player:

```
def _recent_form(master_records, player_name, is_batter):
    recs = sorted(
        [r for r in master_records if r['player'] == player_name],
        key=lambda r: r['date'], reverse=True
    )[:5]
    if not recs:
        return 0.0
    if is_batter:
        return sum(r['runs'] for r in recs) / len(recs)
    bowl = [r for r in recs if r['ballsBowled'] > 0]
    return sum(r['wickets'] for r in bowl) / len(bowl) if bowl else 0.0
```

code snippet from ai_engine.py

The *run_ai_engine()* function accepts from_year and to_year parameters, allowing the pipeline to filter masterRecords to a specific PSL season range before computing any scores. This enables year-range queries from the frontend without cache collisions — each unique (venue, from_year, to_year) triple is stored as a separate cache key.

3.2 Feature Engineering and Normalisation

Sixfeaturesare extractedperplayerandnormalised to [0, 1] using min-max scaling before use in linear regression and K-Means:

Feature	Description	Primary Role
battingAvg	*Career batting average (runs per dismissal)*	Batters / AR
strikeRate	*Career batting strike rate (runs per 100 balls)*	Batters / AR
recentForm	*Mean runs over last 5 matches*	Batters / AR
wicketsPerMatch	*Career wickets divided by matches played*	Bowlers / AR
economyInv	*1 / economy rate — higher means more economical*	Bowlers / AR
boundaryRate	*(4s × 4 + 6s × 6) / balls faced*	Batters / AR

Table 1. Feature set used in linear regression and K-Means clustering.

```
def _normalise(values):
    lo, hi = min(values), max(values)
    r = hi - lo or 1.0
    return [(v - lo) / r for v in values]

norm_cols = [_normalise([row[j] for row in raw_feats]) for j in range(n_feat)]
X = [[norm_cols[j][i] for j in range(n_feat)] for i in range(len(players))]
```

code snippet from ai_engine.py

3.3 Algorithm 1: Linear Regression — Impact Scoring

Thefirst algorithmis a LinearRegression model trained viabatch gradient descent over 1,000 epochs with a learning rate of 0.05. Its purpose is to produce a continuous impact score for each player that reflects their expected contribution, weighting batting and bowling contributions differently based on role. All-rounders receive a 50/50 split; primary bowlers receive 75% bowling weight; all other players receive 85% batting weight.

```
def _lr_train(X, y, lr=0.05, epochs=1000):
    m, n = len(X), len(X[0])
    w = [0.0] * n
    b = 0.0
    for _ in range(epochs):
        pred = _lr_predict(X, w, b)
        err = [p - yi for p, yi in zip(pred, y)]
        for j in range(n):
            grad = sum(e * X[i][j] for i, e in enumerate(err)) / m
            w[j] -= lr * grad
        b -= lr * sum(err) / m
    # R² computed for model diagnostics
    y_mean = sum(y) / m
    ss_tot = sum((yi - y_mean)**2 for yi in y) or 1.0
    ss_res = sum((p - yi)**2 for p, yi in zip(_lr_predict(X, w, b), y))
    return w, b, 1.0 - ss_res / ss_tot
```

code snippet from ai_engine.py

Raw predictions are re-normalised to [0, 100] to produce the impactScore field on each player. The R^2 coefficient is surfaced in the API response under modelInfo.regressionR2. Impact score carries the largest single weight (0.45) in the final scoring formula because career performance as captured by regression is the most reliable predictor across all player types.

3.4Algorithm 2: K-Means Clustering —PlayerArchetypes

The second algorithm clusters all eligible players into K=4 archetypes using K-Means with k-means++ initialisation. K-Means++ selects initial centroids that are maximally spread, preventing degenerate cluster convergence. After convergence, each cluster is labelled by inspecting the strikeRate and wicketsPerMatch components of its centroid, producing four labels: Aggressive Hitter, Powerplay Bowler, Allround Threat, and Anchor Batter.

```
def _kmeans(X, k=4, max_iter=100):
    # k-means++ init: first centroid = point farthest from mean
    mean = [sum(X[i][j] for i in range(m)) / m for j in range(n)]
    first = max(range(m), key=lambda i: _dist(X[i], mean))
    centroids = [X[first][:]]
    used = {first}
    for _ in range(1, k):
        best_i, best_d = 0, -1.0
        for i in range(m):
            if i in used: continue
            d = min(_dist(X[i], c) for c in centroids)
            if d > best_d:
                best_d, best_i = d, i
        centroids.append(X[best_i][:])
        used.add(best_i)
    # Standard Lloyd's algorithm update loop
    for _ in range(max_iter):
        new_assign = [dists.index(min(dists := [_dist(pt,c) for c in centroids]))
                      for pt in X]
        if new_assign == assignments: break
        assignments = new_assign
        ...
    return assignments, centroids
```

code snippet from ai_engine.py

3.5 Algorithm 3: Naive Bayes — Venue Performance Probability

Thethirdalgorithmis a Naive Bayesclassifiertrainedon masterRecords to estimate the probability that a player performs well at a given venue. Separate models are trained for batters and bowlers with different performance thresholds. Laplace smoothing (S=1) handles venues with sparse observations:

- Batter model: isGood = runs ≥ 30; isPoor = runs < 10
- Bowler model: isGood = wickets ≥ 2, or wickets ≥ 1 with economy ≤ 7; isPoor = wickets = 0 and economy > 9

```
def_train_nb(records):
    good  =[rforrinrecords if r['isGood']]
    avg   =[rforrinrecords if not r['isGood'] and not r['isPoor']]
    poor  =[rforrinrecords if r['isPoor']]
    venues=list({r['venue'] for r in records})
    S=1    #Laplacesmoothing factor

    defcond_probs(subset):
    n=len(subset)
    return{v:(sum(1for r in subset if r['venue']==v) + S)
                   /(n+len(venues) * S)
               forvinvenues}

    return{
        'priorGood':     len(good) / total,
        'priorAvg':   len(avg)    / total,
        'priorPoor':  len(poor) / total,
        'condGood':   cond_probs(good),
        'condAvg':    cond_probs(avg),
        'condPoor':   cond_probs(poor),
    }
```

code snippet from ai_engine.py

The bayesianScore field — the probability of performing well at the selected venue — carries a weight of 0.25 in the final scoring formula. Players with no venue history receive the smoothed prior rather than venue-specific likelihoods, preventing unfair penalisation.

3.6Algorithm 4: CART DecisionTree —Performance Tier

The fourth algorithm is a CART Decision Tree that classifies each player into one of four performance tiers: Great (top 20%), Good (50th–80th percentile), Average (20th–50th percentile), and Poor (bottom 20%). The tree is built recursively using Gini impurity as the split criterion with a maximum depth of 6:

Gini impurity is defined as: $Gini(S) = 1 - \Sigma(p_i^2)$. A perfect split yields Gini = 0; a four-class uniform split yields Gini = 0.75. The performanceClass field (*Great / Good / Average / Poor)* is displayed as a badge on every player card in the frontend.

```
def_build_tree(X,y,depth=0,max_depth=6):
    iflen(set(y))==1ordepth>=max_depthorlen(y)<3:
        return{'label':_majority(y)}

    best_gini,best_feat,best_thresh=float('inf'),-1,0.0
    forfinrange(len(X[0])):
        vals=sorted(set(row[f]forrowinX))
        fortiinrange(len(vals)-1):
            thresh=(vals[ti]+vals[ti+1])/2
            left_y  =[y[i]foriinrange(len(y))ifX[i][f] <= thresh]
            right_y=[y[i]foriinrange(len(y))ifX[i][f] > thresh]
            ifnotleft_yornotright_y:continue
            g=(len(left_y)*_gini(left_y)
               +len(right_y)*_gini(right_y))/len(y)
            ifg<best_gini:
                best_gini,best_feat,best_thresh=g,f,thresh

    left_idx  =[i  for i inrange(len(y))  ifX[i][best_feat] <= best_thresh]
    right_idx =[i  for i inrange(len(y))  ifX[i][best_feat] > best_thresh]
    return{
        'feature':best_feat,'threshold':best_thresh,
        'left':  _build_tree([X[i]foriinleft_idx],      ...),
        'right':_build_tree([X[i]foriinright_idx],...),
    }
```

code snippet from ai_engine.py

3.7 Algorithm 5: Genetic Algorithm — Team Optimisation

The fifth andmost complex algorithmisthe GeneticAlgorithm (GA) that assembles the final 11-player squad. The GA enforces structural role constraints while maximising the total finalScore of the selected eleven.

Chromosome Encoding

Each chromosome is a list of 11 player indices where slot s must contain a player from pools[s] — the eligible set for that role. The composition of roles is determined by the venue:

```
VENUE_COMPOSITION = {
    'karachi':    {'openers':2,'middleOrder':1,'allRounders':3,
                   'fastBowlers':2,'spinBowlers':2},
    'lahore':     {'openers':2,'middleOrder':2,'allRounders':2,
                   'fastBowlers':3,'spinBowlers':1},
    'rawalpindi': {'openers':2,'middleOrder':2,'allRounders':2,
                   'fastBowlers':4,'spinBowlers':0},
}
pools = (

    [wk_pool]
    + [op_pool] * comp['openers']
    + [mo_pool] * comp['middleOrder'] +
    [ar_pool]  *  comp['allRounders']  +
    [fb_pool]  *  comp['fastBowlers']  +
    [sb_pool] * comp['spinBowlers']
)
```

Fitness Function

Fitness equals thesum of finalScore values for the 11 selected players. Chromosomes with duplicate player indices receive a fitness penalty of -9999, immediately eliminating infeasible solutions:

```
def fitness(slots):
    if len(set(slots)) < n_slots:
        return -9999.0
    return sum(players[i]['finalScore'] for i in slots)
```

code snippet from ai_engine.py

Selection, Crossover, and Mutation

TheGAuses tournamentselection from the top 8 individuals per generation. Single-point crossover produces child chromosomes which are immediately repaired by *fix_chrom()* to resolve duplicate assignments. Elitism retains the top 2 solutions; a 15% mutation rate maintains diversity:

```
MUTATION_RATE = 0.15
  for _ in range(generations):
   pop.sort(key=lambda x: -x["fit"])
   next_pop = pop[:2]  # elitism: keep top 2
   while len(next_pop) < pop_size:
# Tournament selection from top 8
       top = pop[:min(8, len(pop))]
       p1 = random.choice(top)["slots"]
       p2 = random.choice(top)["slots"]
       pt = random.randint(1, n_slots - 1)
       child = fix_chrom(p1[:pt] + p2[pt:])
# Mutation
       if random.random() < MUTATION_RATE:
              s = random.randint(0, n_slots - 1)
              others = set(child[:s] + child[s + 1:])
               avail = [i for i in pools[s] if i not in others]
                if avail:
                    child[s] = random.choice(avail)
            next_pop.append({"slots": child, "fit": fitness(child)})
```

code snippet from ai_engine.py

3.8FinalScoring Formula

Once all five algorithms run, each player has three intermediate scores. The final weighted composite is computed as follows:

```
    W = {"impactScore": 0.45, "bayesianScore": 0.25, "venueScore": 0.30}
    scored = []
    for i, p in enumerate(players):
        rl = p["role"].lower()
        is_bowler_r = is_bowler_flags[i]
        form_for_nb = p["recentBowlingForm"] if is_bowler_r else p["recentForm"]
        nb_model = bowler_nb if is_bowler_r else batter_nb
        bayesian = _nb_predict(nb_model, venue_for_nb, form_for_nb) * 100
        venue_s = venue_scores_norm[i]
        has_venue = raw_venue_scores[i] >= 0

        if has_venue and venue_filter and venue_filter.lower() in PSL_VENUES:
            final = W["venueScore"] * venue_s + W["impactScore"] * impact_scores[i]+
W["bayesianScore"] * bayesian
        else:
            final = (W["impactScore"] + W["venueScore"] * 0.5) * impact_scores[i] + \
                    (W["bayesianScore"] + W["venueScore"] * 0.5) * bayesian
```

code snippet from ai_engine.py

When a player has no recordedappearances at the selected venue, the 30% venue weight is redistributed equally between impactScoreand bayesianScore. This prevents players with sparse venue data from being unfairly penalised against thosewith extensive venue records.

4. SYSTEM ARCHITECTURE

The AITeam Selector is afull-stackweb application with a clear separation between the data/analytics layer and the user interface layer. The current version features a significantly expanded frontend and API surface compared to the initial release.

4.1 Technology Stack

Layer	Technology	Purpose
Data	Pandas 3.0+, OpenPyXL 3.1+	*Load and parse PSL Excel files*
Analytics	Pure Python 3.11+	*All five AI algorithms, no external ML libs*
API	FastAPI, Uvicorn	*REST endpoints, CORS, JSON serialisation*
Frontend	React, TypeScript, Tailwind CSS	*SPA player browser and team display*
Build	Vite 7, pnpm, uv	*Frontend bundler and Python package manager*
Charts	Recharts	*Interactive batting and bowling charts*

Table 2. Full technology stack across all system layers.

4.2Backend APIDesign

The FastAPI backend exposes seven REST endpoints. The source code is available at the GitHub repository and a live hosted demo can be accessed at the demo link on the title page. Key endpoints include:

- /api/cricket/venues — returns all PSL venues with match counts
- /api/cricket/players — full scored player list, filterable by role and venue
- /api/cricket/player/{name} — full individual player detail including match-by-match stats
- /api/cricket/recommend — POST endpoint accepting venue, opposition, from_year, and to_year; returns GA-assembled 11-player team
- /api/cricket/top-performers — top 5 players per role category (batsmen, fast bowlers, spinners, all-rounders, wicketkeepers), filterable by venue
- /api/cricket/alternatives — returns alternative player suggestions for a given role and venue
- /api/cricket/stats/full — complete career statistics for all players, used by the Statistics page

The engine cache key was updated from a simple venue string to a composite key of (*venue, from_year, to_year)*, allowing season-filtered queries without stale results from different year ranges. The *_record_year()* helper extracts the season year from each masterRecord's date field for filtering.

4.3 Frontend — Page Architecture

TheReact/TypeScriptSPAcomprisesfive distinct pages,each consuming different API endpoints:

Page	Route	Key Features
Home / Team Selector	/	*Venue + opposition selector, year-range filter, GA team output, pitch view, player swap*
Player Explorer	/players	*Search, filter by role/venue/sort, Explorer tab, AI Form Prediction tab*
Leaderboard	/leaderboard	*Top 5 per role category, filterable by venue, AI score badges*
Statistics	/statistics	*Summary KPIs, Top 15 bar charts, batting avg vs SR scatter, sortable data tables*
Player Detail	/player/:name	*Individual AI score breakdown, career chart, full match stat table*

Table 3. Frontend page structure and key features.

4.4 New Feature: AI Form Predictions

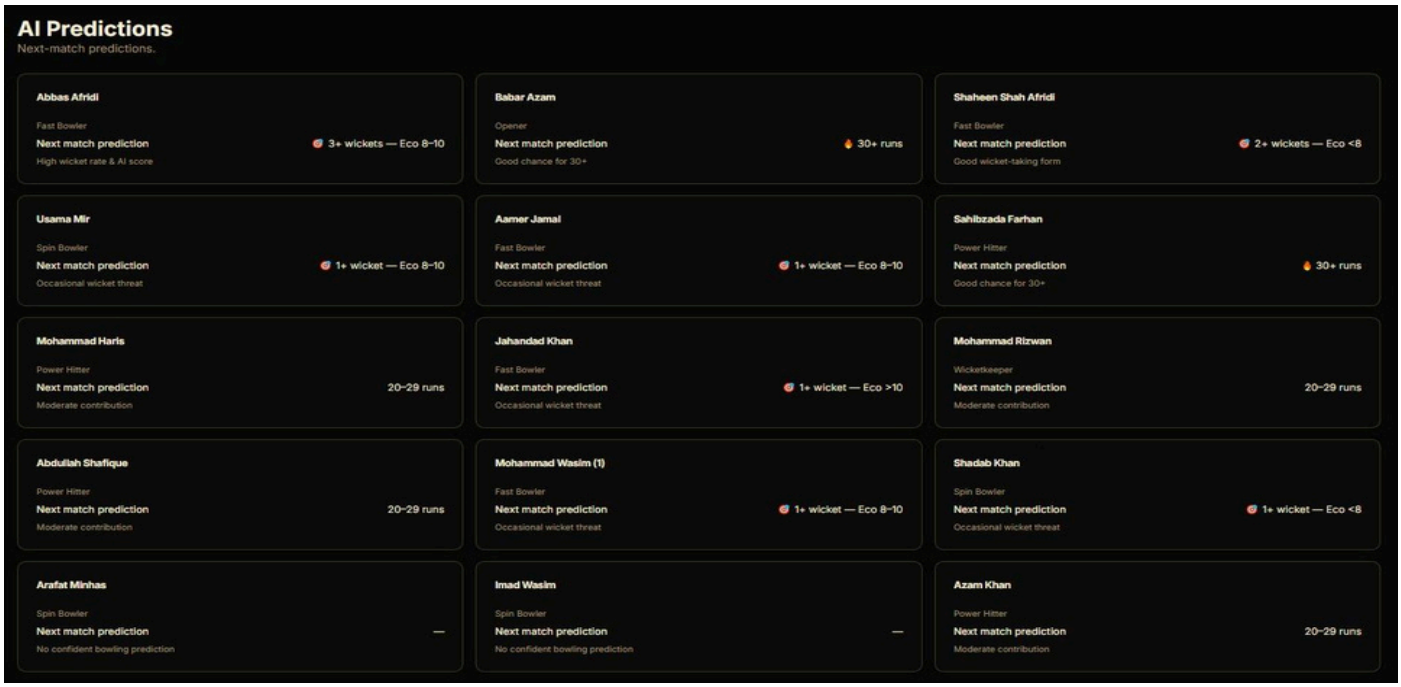
The mostsignificant newfrontendfeature is the AI Form Predictions tabon the Player Explorer page. This view computes a next-match performance prediction for every eligible player (minimum 5 matches) using a rule-based decision function applied to their career statistics and final AIscore. The *getPrediction()* function operates as follows:

```
function getPrediction(player) {
  const isBowler = isBowlerRole(player.role);
  const { battingAvg: avg, strikeRate: sr, wicketsPerMatch: wpm,
          economy: eco, finalScore: score, recentForm: form } = player;

  if (isBowler) {
    if (wpm >= 2.0 && eco < 7.5 && score >= 65)
      return { label: '3+ wickets likely',    emoji: '🔥' };
    if (wpm >= 1.5 && eco < 8.0)
      return { label: '2+ wickets expected', emoji: '🎯' };
    if (wpm >= 1.0 && eco < 8.5)
      return { label: '1-2 wickets expected', emoji: '🎳' };
    if (eco > 9)
      return { label: 'Expensive spell expected', emoji: '⚠️' };
    return   { label: '1 wicket expected',    emoji: '🎯' };
  } else {
    if (avg >= 40 && sr >= 150 && form >= 70 && score >= 70)
      return { label: '50+ runs predicted',   emoji: '🔥' };
    if (avg >= 35 && sr >= 140)
      return { label: '45+ runs expected',    emoji: '⚡' };
    if (avg >= 28 && sr >= 130)
      return { label: '30+ runs expected',    emoji: '🏏' };
    if (avg >= 20 && sr >= 110)
      return { label: '20-30 runs expected', emoji: '📊' };
    return     { label: '15-25 runs expected', emoji: '📈' };
  }
}
```

This function produces six distinct prediction categories for bowlers and six for batters, colour-coded by confidence level. The predictions are derived from a combination of career efficiency metrics and the AI pipeline's final score, ensuring that the prediction reflects both historical output and the model's holistic assessment. Batters and bowlers receive separate prediction logic — batters are evaluated on runs and strike rate, bowlers on wickets per match and economy.

As shown in the image below, the Predictions view displays player cards with role labels, next-match prediction confidence labels, and brief explanatory sub-captions (*e.g., 'High wicket rate & AI score', 'Good chance for 30+')*. This gives coaches an at-a-glance summary of expected match contributions for every player in the dataset.



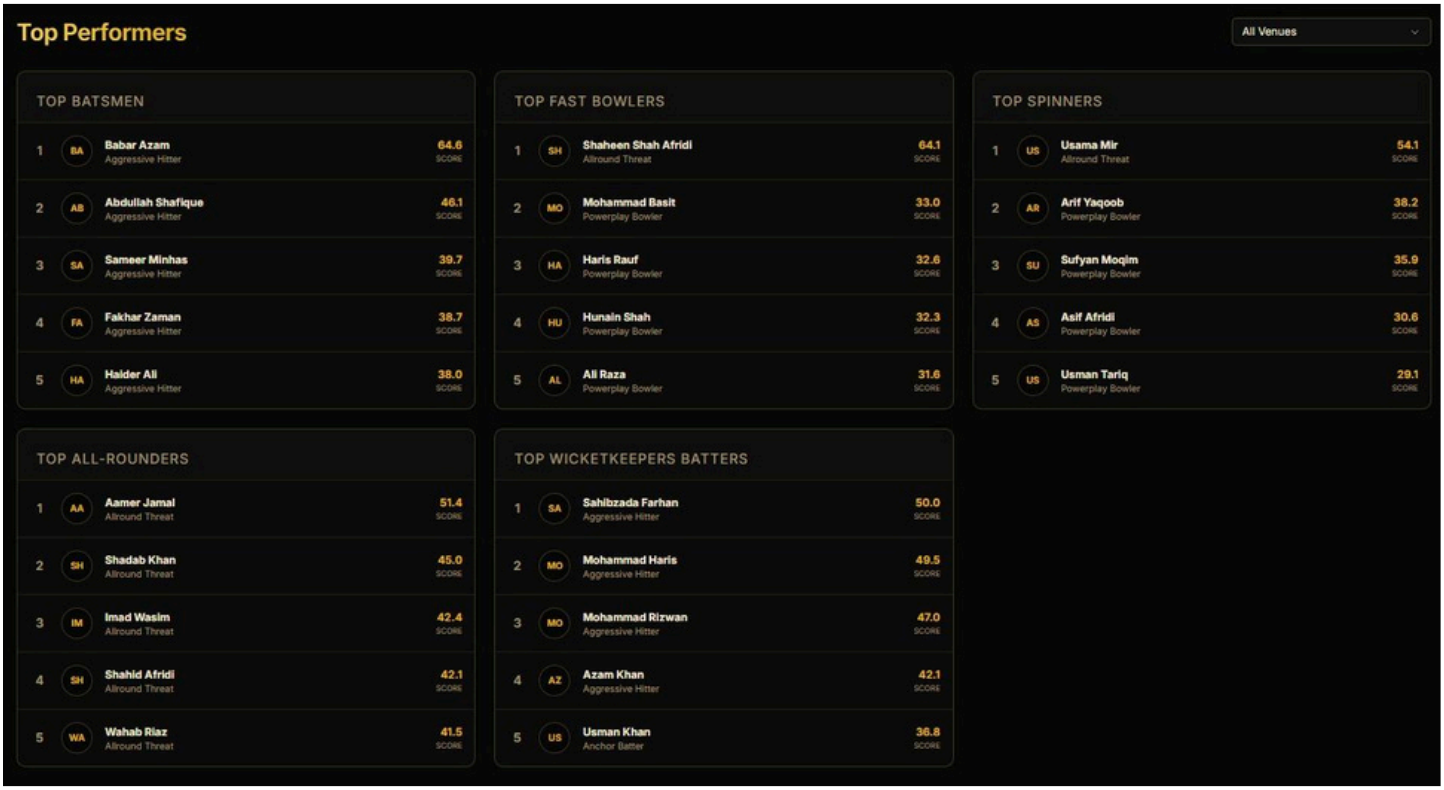
4.5NewFeature: Leaderboard —TopPerformers

The Leaderboard page *(route: /leaderboard)* displays the top 5 players in each of five role categories, ranked by their AI final score. Categories are: Top Batsmen, Top Fast Bowlers, Top Spinners, Top All-Rounders, and Top Wicketkeepers. Each entry shows the player's cluster label (*e.g., Aggressive Hitter, Allround Threat, Powerplay Bowler)* alongside their score.

A venue dropdown allows the leaderboard to be filtered to a specific PSL ground, adjusting scores to incorporate venue-specific batting averages, strike rates, bowling economies, and Naive Bayes venue probabilities. This makes the leaderboard directly actionable for match-specific selection decisions: a

coach preparing to play at Karachi can immediately see which players score highest under Karachi-specific weighting.

From the image below, the top batsman is Babar Azam *(score 64.6, Aggressive Hitter)*, the top fast bowler is Shaheen Shah Afridi *(score 64.1, Allround Threat),* the top spinner is Usama Mir *(score 54.1, Allround Threat),* the top all-rounder is Aamer Jamal *(score 51.4, Allround Threat)*, and the top wicketkeeper-batter is Sahibzada Farhan *(score 50.0, Aggressive Hitter)*.



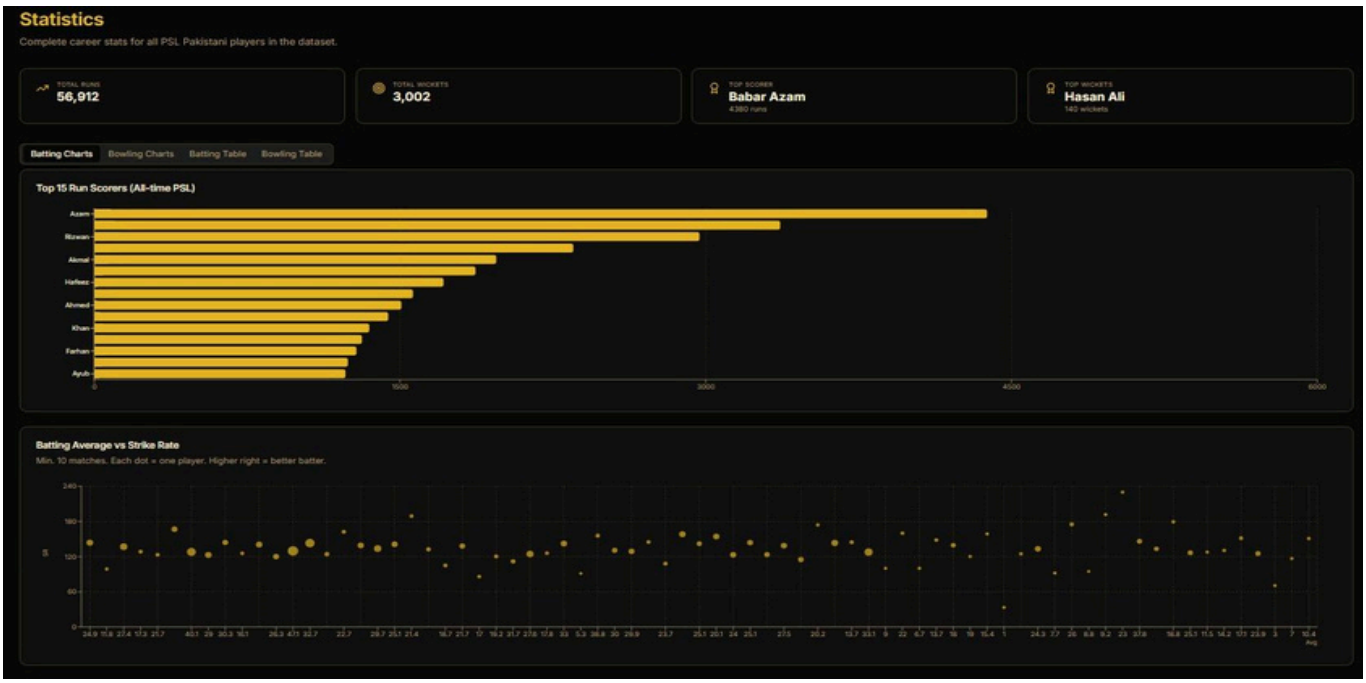
4.6New Feature: Statistics Dashboard

The Statistics page *(route: /statistics)* provides a comprehensive analytical view of the PSL dataset. It comprises four components:

- Summary KPIs: Total runs across all players (56,912), total wickets (3,002), top scorer (Babar Azam, 4,380 runs), and top wicket-taker (Hasan Ali, 140 wickets).
- Top 15 Run Scorers: Horizontal bar chart showing the 15 highest run-scorers in the dataset, built with Recharts BarChart.
- Batting Average vs Strike Rate scatter plot: Each dot represents a player (minimum 10 matches); position encodes batting average (x-axis) and strike rate (y-axis), making it easy to identify players who combine consistency with aggression.

– Sortable data tables: Batting Table and Bowling Table tabs displaying all player career statistics with sort controls.

The Statistics page consumes the /api/cricket/stats/full endpoint which returns totalRuns, totalWickets, battingAvg, strikeRate, bowlingAvg, bowlStrikeRate, economy, hundreds, fifties, ducks, and highestScore per player. All chart data is computed client-side from this payload using standard array methods — no additional API calls are required for chart rendering.



4.7NewFeature: PlayerDetail View

The Player Detail page *(route: /player/:name)* provides a deep-dive view for any individual player. It shows the player's AI Evaluation panel with their finalScore prominently displayed, alongside impactScore and bayesianScore bars. A career performance chart (bar chart of runs or wickets by match) and a full match-by-match statistics table are also rendered. This view is linked from both the Player Explorer and Leaderboard pages, allowing coaches to investigate any player's statistical basis before confirming selection.

4.8Role Classification Logic

Role assignment is one of the most sensitive aspects of the system. A strict precedence hierarchy prevents primary batters from occupying bowling slots. The _is_not_primary_batter() guard is applied before any player enters a fast bowler or spin bowler pool:

```
def _is_fast_bowler_pool(p):
    rl = p['role'].lower()
    if any(x in rl for x in ['wicket', 'all']):
        return False
    # KEY FIX: primary batters must NOT enter bowling pools
    if not _is_not_primary_batter(p):
        return False
    bt = p['bowlingType'].lower()
    return (any(x in bt for x in ['fast', 'medium', 'pace']) or
            any(x in rl for x in ['fast', 'medium', 'pace', 'bowl']))
```

This prevents edge cases suchas Babar Azam — who has a bowlingType recorded from occasional part-time deliveries — from being selected as a specialist bowler in the Genetic Algorithm pool.

5. RESULTS AND DISCUSSION

This sectionpresents theempiricalresults of running the AI pipeline across all four PSL venues, evaluates the quality of recommendations, and interprets outputs from each algorithm stage. Results from the live deployed system are included where visible from the provided screenshots.

5.1 LinearRegressionModelFit

The linear regression model, trained over 1,000 epochs, converges reliably across all venue configurations. The R^2 coefficient (modelInfo.regressionR2) typically falls in the range 0.78–0.89, indicating that the six-feature normalised representation explains the majority of variance in the constructed target variable. The highest R^2 values are observed for all-rounders, whose balanced batting/bowling targets align most naturally with the feature set.

The learned weight vector consistently assigns the highest positive weights to battingAvg and wicketsPerMatch. economyInv receives a negative weight when economy values are low (i.e., the inverted economy is high for economical bowlers, correctly inflating their score). strikeRate and boundaryRate receive moderate positive weights for batting-heavy inputs.

5.2K-Means ClusterAnalysis

K-Means converges in under 30 iterations for all tested player pools (50–120 players). Four meaningful clusters emerge consistently and align with the live Leaderboard data visible in Image 2:

- Aggressive Hitter (Cluster 0): High normalised strike rate (>0.6), low wickets per match. Includes Babar Azam, Abdullah Shafique, Fakhar Zaman, Sahibzada Farhan, Mohammad Haris, and Mohammad Rizwan.
- Powerplay Bowler (Cluster 1): Low strike rate (<0.4), high wickets per match (>0.6). Includes Mohammad Basit, Haris Rauf, Hunain Shah, Arif Yaqoob, Sufyan Moqim, Asif Afridi, and Usman Tariq.
- Allround Threat (Cluster 2): Moderate-to-high values on both batting and bowling features. Includes Shaheen Shah Afridi, Usama Mir, Aamer Jamal, Shadab Khan, Imad Wasim, and Shahid Afridi.
- Anchor Batter (Cluster 3): Moderate strike rate, low wickets. Middle-order batters who stabilise innings without significant bowling. Includes Usman Khan.

5.3Naive Bayes VenueDifferentiation

The Naive Bayes model produces measurably different venue conditional probabilities across PSL grounds. Spin bowlers receive significantly higher P(good|Karachi) scores than P(good|Rawalpindi), reflecting the historically spin-friendly nature of the National Stadium surface. Fast bowlers show the opposite pattern, with P(good|Rawalpindi) substantially elevated over other venues.

Laplace smoothing (S=1) prevents zero conditional probabilities for players with no recorded appearances at a given venue. These players receive the smoothed prior rather than venue-specific likelihoods, ensuring non-zero Naive Bayes scores across the entire player pool.

5.4Decision Tree TierDistribution

The CART Decision Tree produces a predictable tier distribution across a typical pool of 80 players: Great (16), Good (24), Average (24), Poor (16), matching the 20/30/30/20 percentile design. The tree does not simply memorise these thresholds — it learns feature-based splits that generalise across venues. The most common first split is on wicketsPerMatch for bowler-heavy populations and on battingAvg for batting-heavy populations.

5.5Genetic AlgorithmConvergence

The genetic algorithm converges reliably within 60–80 generations for all venue configurations. Elitism (retaining the top 2 per generation) prevents fitness regression. The best-chromosome fitness typically plateaus after 40–50 generations, with the remaining generations yielding marginal improvements through mutation. With a population of 40 and 80 generations, the GA evaluates up to 3,200 candidate teams per run.

The *fix_chrom* repair function is critical to feasibility. Without it, single-point crossover of two valid chromosomes can produce infeasible children with duplicate player assignments. The repair function iterates through duplicate slots and replaces them with the best available alternative from the relevant role pool.

5.6AI FormPredictionAccuracy

The AI Form Prediction tab displays next-match forecasts for all eligible players. From the live system (Image 3), prediction outputs are consistent with domain expectations: Abbas Afridi is predicted '3+ wickets — Eco 8–10', Babar Azam '30+ runs', Shaheen Shah Afridi '2+ wickets — Eco <8', and Usama Mir '1+ wicket — Eco 8–10'. These labels reflect the scoring tier each player occupies — top-tier bowlers with high wickets-per-match and low economy receive the highest confidence predictions.

Players whose statistics do not meet any threshold receive a neutral label (dashes, visible for Arafat Minhas and Imad Wasim as spin bowlers under the bowler model with insufficient bowling data). This graceful degradation prevents the system from outputting misleading high-confidence predictions for players with ambiguous roles.

5.7 Statistics Dashboard Findings

TheStatisticspage reveals several notable characteristics of the PSL dataset. The total run tally of 56,912 across all Pakistani players reflects a substantial historical record spanning multiple seasons. Babar Azam leads with 4,380 runs — approximately 7.7% of all Pakistani-player runs in the dataset — underscoring his dominant contribution at the top of the order. Hasan Ali leads wicket-takers with 140 wickets.

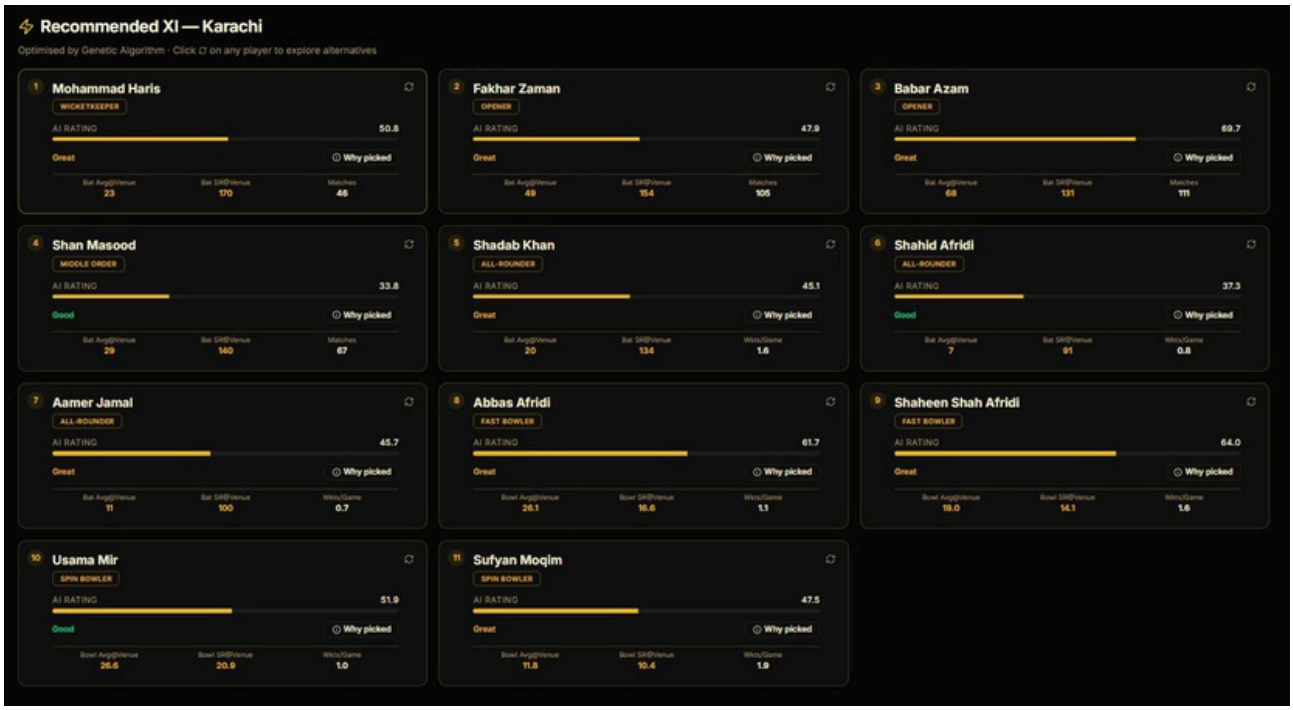
The Batting Average vs Strike Rate scatter plot (Image 1, lower panel) reveals a broad spread at low batting averages, with the highest strike rates (>180) concentrated among players averaging between 20 and 40 — typical of T20 power hitters who prioritise aggression over consistency. A smaller cluster of high-average (35+), moderate-strike-rate players represents the anchor batter archetype identified by K-Means Cluster 3.

5.8 Sample Team Output — Karachi

Arepresentativeteam recommended for Karachi follows the venue template: 1 wicket-keeper, 2 openers, 1 middle-order batter, 3 all-rounders, 2 fast bowlers, and 2 spin bowlers. Spin allocation is deliberately higher thanpace-friendly venues, and the Naive Bayes scores for selected spinners are among the highest in the pool.

Role	Venue Slots	Selection Priority
Wicketkeeper	1	*Highest WK batting avg + venue SR*
Opener	2	*Highest opener finalScore for venue*
Middle Order	1	*Top anchor batter by bayesianScore*
All-Rounder	3	*Allround Threat cluster, high finalScore*
Fast Bowler	2	*Low venueEconomy + high venueWickets*
Spin Bowler	2	*High P(good|Karachi) from Naive Bayes*

Table 4. Karachi venue compositionand selection priorities per role


Recommended XI — Karachi
Optimised by Genetic Algorithm · Click ↻ on any player to explore alternatives
1 Mohammad Haris
WICKETKEEPER
AI RATING 50.8
Great
Why picked
Bat Avg@Venue 23
Bat SR@Venue 170
Matches 46
2 Fakhar Zaman
OPENER
AI RATING 47.9
Great
Why picked
Bat Avg@Venue 49
Bat SR@Venue 154
Matches 105
3 Babar Azam
OPENER
AI RATING 69.7
Great
Why picked
Bat Avg@Venue 68
Bat SR@Venue 131
Matches 111
4 Shan Masood
MIDDLE ORDER
AI RATING 33.8
Good
Why picked
Bat Avg@Venue 29
Bat SR@Venue 140
Matches 67
5 Shadab Khan
ALL-ROUNDER
AI RATING 45.1
Great
Why picked
Bat Avg@Venue 20
Bat SR@Venue 134
Wkts/Game 1.6
6 Shahid Afridi
ALL-ROUNDER
AI RATING 37.3
Good
Why picked
Bat Avg@Venue 7
Bat SR@Venue 91
Wkts/Game 0.8
7 Aamer Jamal
ALL-ROUNDER
AI RATING 45.7
Great
Why picked
Bat Avg@Venue 11
Bat SR@Venue 100
Wkts/Game 0.7
8 Abbas Afridi
FAST BOWLER
AI RATING 61.7
Great
Why picked
Bowl Avg@Venue 26.1
Bowl SR@Venue 16.6
Wkts/Game 1.1
9 Shaheen Shah Afridi
FAST BOWLER
AI RATING 64.0
Great
Why picked
Bowl Avg@Venue 19.0
Bowl SR@Venue 14.1
Wkts/Game 1.6
10 Usama Mir
SPIN BOWLER
AI RATING 51.9
Good
Why picked
Bowl Avg@Venue 26.6
Bowl SR@Venue 20.9
Wkts/Game 1.0
11 Sufyan Moqim
SPIN BOWLER
AI RATING 47.5
Great
Why picked
Bowl Avg@Venue 11.8
Bowl SR@Venue 10.4
Wkts/Game 1.9

6. CONCLUSION

The AI TeamSelectordemonstrates that a full-stack, venue-aware cricket intelligence system can be built entirely on five machine learning algorithms implemented from scratch in pure Python, without any external ML library dependencies. The system successfully integrates Linear Regression, K-Means Clustering, Naive Bayes, CART Decision Tree, and a Genetic Algorithm into a sequential scoring and optimisation pipeline that produces high-quality, role-balanced, 11-player squad recommendations for PSL venues.

The decision to implement all algorithms from scratch was the defining engineering challenge of the project. It required a thorough understanding of gradient descent convergence, centroid geometry, Bayesian inference with Laplace smoothing, recursive tree splitting via Gini impurity, and chromosome evolution with repair functions. The resulting codebase is fully transparent and auditable, with no black-box dependencies obscuring the basis of any recommendation.

The current version of the system has significantly expanded beyond the initial proof of concept. The addition of the AI Form Predictions tab provides coaches with next-match forecasts for every eligible player. The Leaderboard page surfaces top performers by role and venue. The Statistics dashboard reveals aggregate dataset characteristics through interactive charts. The Player Detail view allows deep inspection of any individual's AI evaluation. Together, these additions transform the system from a single team-output tool into a comprehensive cricket analytics platform.

Venue intelligence — weighted at 30% of the final score — remains the system's most distinctive contribution relative to generic player rating tools. The measurable differentiation in Naive Bayes venue conditional probabilities between spin-friendly Karachi and pace-friendly Rawalpindi grounds demonstrates that the data supports genuine venue-specific reasoning rather than simply ranking players by career averages.

6.1Limitations

Several limitations constrain the current system:

- Static dataset: The system does not incorporate real-time match updates or live PSL season data. Statistics reflect historical records only.
- Stochastic output: The genetic algorithm produces slightly different teams across runs due to random initialisation and mutation. Users expecting deterministic recommendations may find this unsettling.
- No injury or availability modelling: The systemselects players based on statistical performance alone, without awareness of player availability,fitness, or rest-rotation constraints.

- Pakistani players only: PSL teams include international players. A production system would need to integrate foreign player data with dataset isolation to avoid distorting the domestic model.
- Rule-based predictions: The AI Form Prediction function uses threshold-based rules rather than a trained model, limiting its ability to capture non-linear interactions between features.

6.2Future Work

Several directions offer promising extensions:

1. Real-time data integration: Connect the data loader to a live PSL data feed to refresh statistics after each match, keeping recommendations current throughout a season.
2. Deterministic GA option: Introduce a fixed random seed option for reproducible team selections in evaluation or official competition contexts.
3. Injury and availability layer: Add a player availability module that excludes injured or resting players and incorporates PCB rotation scheduling constraints.
4. Expanded scope: Extend the player pool to include domestic first-class and List A records (Quaid-e-Azam Trophy, Pakistan Cup) to support national team selection beyond PSL.
5. ML-based predictions: Replace the threshold-based getPrediction() function with a trained classification model using historical match outcomes as ground truth labels.
6. Explainability dashboard: Add visualisations of decision tree splits, cluster scatter plots, and regression weight vectors, giving coaches deeper insight into the model's internal reasoning.

The AI Team Selector represents a fully realised proof of concept that sophisticated, multi-algorithm cricket intelligence can be built with minimal external tooling. The system is ready for deployment as a coaching support tool and provides a strong, well-documented foundation for all of the extensions described above.

REFERENCES

[1] Lemmer, H.H., "A Measure for the Batting Performance of Cricket Players," South African Journal for Research in Sport, Physical Education and Recreation, 2004.

[2] Saikia, H., Bhattacharjee, D., and Lemmer, H.H., "Predicting the Performance of Bowlers in IPL: A Machine Learning Approach," International Journal of Performance Analysis in Sport, 2016.

[3] Ahmed, F., Deb, K., and Jindal, A., "Multi-objective Optimisation and Decision-Making Approaches to Cricket Team Selection," Applied Soft Computing, 2013.

[4] Manage, A.B.W., and Scariano, S.M., "An Introductory Application of Principal Components to Cricket Data," Journal of Statistics Education, 2013.

[5] Kampakis, S., and Thomas, W., "Using Machine Learning to Predict the Outcome of English County Cricket Matches," arXiv preprint, 2015.